The Sleeping Teaching Assistant

Project Description:

- The Sleeping Teaching Assistant project simulates the interactions between a Teaching Assistant (TA) and students in a university computer students with programming assignments, and students in science department during office hours.
- The TA helps any need to wait if the TA is busy.
- If all TA are busy and no chairs available, students leave and come back later.

Tools:

- The project is implemented using Java threads, mutex locks, GUI ,and semaphores to coordinate the activities of the TA and students.

What we have actually did:

- In the Sleeping Teaching Assistant (TA) problem, we implemented a solution using Java threads, mutex locks, and semaphores to coordinate the activities of TAs and students in a university computer science department . Here's a quick overview of what we did:

- **Class definitions**: Defined classes for student , ta , classroom and manageSession to represent the entities involved in the problem.

- **Concurrency Control:** Implemented synchronization mechanisms such as semaphore , synchronized ,

and reentrantLock to ensure safe concurrent access to shared resources and avoid race conditions.

- **Student and TA Behavior:** Modeled the behavior of students and TAs in the student and ta classes where students can enter the classroom, ask questions, and TAs can answer questions.

- **Session Management:** Managed the overall session using the manageSession class , creating a fixed thread pool to handle concurrent execution of TAs and students.

- **Random Delays:** Introduced random delays between student arrivals and students coming back later to simulate realistic scenarios.

Team members roles:

- TA , Student , manageSession classes → sara ashraf (20210393) , sama gomaa(20210423)
- Classroom class and form gui → sara mahmoud(20210399) , sama Mostafa(20210424)
- Session gui , documentation→ mona mohmed(20210976), Abdelrahman amr mohmed (20210519)

Implementation Details :

TA Class:

1.The “TA” class represents a Teaching Assistant (TA) in the university.

2.The TA is responsible for answering questions from students in the classroom.

3. It implements the Runnable interface to be executed as a separate thread.

4. The main execution method for the TA thread, that the TA continuously answers questions from students in the classroom.

```java
package sleepingta;
/**
 *
 * @author sara mahmoud
 */
public class TA implements Runnable {

    private final Classroom classroom;
    private int TA_ID;

    public TA(Classroom classroom, int TA_ID) {
        this.classroom = classroom;
        this.TA_ID = TA_ID;
    }


    public void setTA_ID(int TA_ID)
    {
        this.TA_ID=TA_ID;
    }

    public int getTA_ID()
    {
        return TA_ID;
    }

    @Override
    public void run(){

        while (true) {
            classroom.answerQuestion(TA_ID);
        }
    }
    }
```

Student Class:

1.The “Student” class represents an student in the university.

2. Students can ask questions during office hours by entering the classroom.

3. It implements the Runnable interface to be executed as a separate thread.

4.The main execution method for the Student thread , that the student asks a question by entering the classroom and It is synchronized to ensure proper thread safety.

```java
package sleepingta;
/**
 *
 * @author sara mahmoud
 */
public class Student implements Runnable {
    private  int studentId;
    private final Classroom classroom;

    public Student(Classroom classroom) {

        this.classroom = classroom;
    }

    public int getStudentId() {
        return studentId;
    }
    public void setStudentId(int studentId)
    {
        this.studentId=studentId;
    }


    @Override
    public void run() {
        try {
            askQuestion();
        } catch (InterruptedException ex) {
        }
    }

    private synchronized void askQuestion() throws InterruptedException {
        classroom.enterClassroom(this);
    }
}
```

Classroom Class:

1.Here is an overview and attributes of Classroom class:

.The `Classroom` class represents the office hours setting where students can ask questions and teaching assistants (TAs) can provide assistance. It coordinates the interaction between .

.students and Tas using semaphores, mutex locks, and synchronized methods.

.The class is responsible for managing the available TAs, chairs, and handling the flow of students.

```java
package sleepingta;
import java.util.logging.Level;
import java.util.logging.Logger;
import java.util.ArrayList;
import java.util.Random;
import java.util.concurrent.Semaphore;
import java.util.concurrent.locks.ReentrantLock;

public class Classroom {

    private final ArrayList<Student> students;
    private final ArrayList<Student> studentsBackLater;
    private final int maxChairs;
    private final int numberOfTA;
    private final int availableTA;
    private int totalAnsweredQuestions, studentBackLaterCounter;
    private final Semaphore available;
    private final ReentrantLock mutex = new ReentrantLock();
    private final Random random = new Random();
    private final Session form;

    public Classroom(int numberOfTA, int numberOfStudent,int maxChairs, Session form) {
        this.maxChairs = maxChairs;
        this.numberOfTA = numberOfTA;
        this.availableTA = numberOfTA;
        this.available = new Semaphore(availableTA);
        this.students = new ArrayList<>(numberOfStudent);
        this.studentsBackLater = new ArrayList<>();
        this.form = form;
```

1.enterClassroom method:

- This method Allows a student to enter the classroom and ask a question.
- Handles waiting students if TAs are busy , as if number of students equal max number of chairs students leaves and come back later, if there is available Tas and student ask question ,if no Tas available and there's available chairs in waiting room students take chair here .

```java
public void enterClassroom(Student student) throws InterruptedException {
    System.out.println("Student " + student.getStudentId() +
            " wants to enter the classroom to ask a question");

    synchronized (students) {
        if (students.size() == maxChairs) {
            System.out.println("No chairs available for Student " + student.getStudentId() +
                    " so Student leaves and will come back later");

            studentsBackLater.add(student);
            mutex.lock();
            try {
                studentBackLaterCounter++;
            } finally {
                mutex.unlock();
            }
        } else {
            students.add(student);

            if(available.availablePermits() == 0) {
                form.TakeChair();
                System.out.println("All TAs are busy so Student " +
                        student.getStudentId() +
                        " takes a chair in the waiting room");

                // Notify waiting threads
                students.notifyAll();

                // Wait for a TA to become available or another student to leave
                students.wait();
            }

            // If a TA becomes available, student can enter the classroom there's at least one
            else if (available.availablePermits() > 0) {
                System.out.println("Student " + student.getStudentId() + " enters the classroom");
                students.notifyAll();
            }
```

2.answerQuestion method:

- This method allows a TA to answer a question from a student.
- Handles synchronization and signaling.
- The method starts by synchronizing on student list , it use while
- loop to wait until there is at least one student in list.
- If there are no students, the TA sleeps.
- The TA thread releases the lock on the list and enters a waiting state.
- If there are still students in the list after answering a question , the TA calls another student to enter the classroom.
- The TA releases the semaphore (**available.release()**) to signal that a TA is now available

```java
public void answerQuestion(int TA_ID) {
    Student student;

    synchronized (students) {
        // Wait until there's at least one student in the list
        while (students.isEmpty()) {
            form.SleepTA(TA_ID);
            System.out.println("TA " + TA_ID + " is waiting for students and sleeps");
            try {

                students.wait();
            } catch (InterruptedException ex) {
            }
        }

        // Get the first student from the list
        student = students.remove(0);
        System.out.println("Student " + student.getStudentId()
                + " finds TA available and asks a question to TA " + TA_ID);

    }
    try {
        // Acquire the semaphore (availableTA) to determine the number of TAs working
        if (available.tryAcquire() && students.size() == maxChairs) {
            available.acquire();
        }

        form.BusyTA(TA_ID);
        System.out.println("TA " + TA_ID + " is answering the question for student "
                + student.getStudentId());
        int workingTime = random.nextInt(2000) + 4000;
        Thread.sleep(workingTime); // Simulate time spent answering a question

        System.out.println("Completed answering question for student "
                + student.getStudentId() + " by TA " + TA_ID
                + " in " + (int) (workingTime / 1000) + " seconds.");
```

• Continue answerQuestion method:

- If there are still students in the list after answering a question

, the TA calls another student to enter the classroom.

- The TA releases the semaphore (**available.release()**) to

signal that a TA is now available

```
        // Acquire the lock for updating shared counters
        mutex.lock();
        try {
            totalAnsweredQuestions++;
        } finally {
            mutex.unlock();
        }
        if (!students.isEmpty()) {
            System.out.println("TA " + TA_ID
                    + " Calls a Student to enter Classroom ");
            form.ReturnChair(TA_ID);
        }

        // Release the semaphore (availableTA)
        available.release();

    } catch (InterruptedException e) {
    }
}
```

3.backLater method:

- I will use it latter in manageSession class to add student that has no chair in waiting room in this Array_list.

```
    public ArrayList<Student> backLater() {
        return studentsBackLater;
    }
}
```

ManageSession Class:

- The **start** method initiates the session, creating a fixed thread pool and a classroom instance.
- Threads are started for both TAs and students, with a random delay between student arrivals.
- Awaits termination of all threads or until 20 seconds pas
- Prints a message when the classroom is closed.

urce History

```java
package sleepingta;


import java.util.ArrayList;
import java.util.Random;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

import static java.util.concurrent.TimeUnit.SECONDS;
import java.util.logging.Level;
import java.util.logging.Logger;

public class manageSession {
    private final int TAs;
    private final int chairs;
    private final int students;

    public manageSession(int TAs, int students, int chairs) {
        this.TAs = TAs;
        this.chairs = chairs;
        this.students = students;
    }

     public void Start(Session form) throws InterruptedException{
        ExecutorService exec = Executors.newFixedThreadPool(12);
        Classroom classroom = new Classroom(TAs, students, chairs, form);
        Random r = new Random();

        System.out.println("Classroom is opened with "+TAs+" TAs");



        for (int i = 1; i <= TAs; i++) {
            TA ta = new TA(classroom, i);
            Thread thTA = new Thread(ta);
            exec.execute(thTA);
        }
```

Continue ManageSession Class:

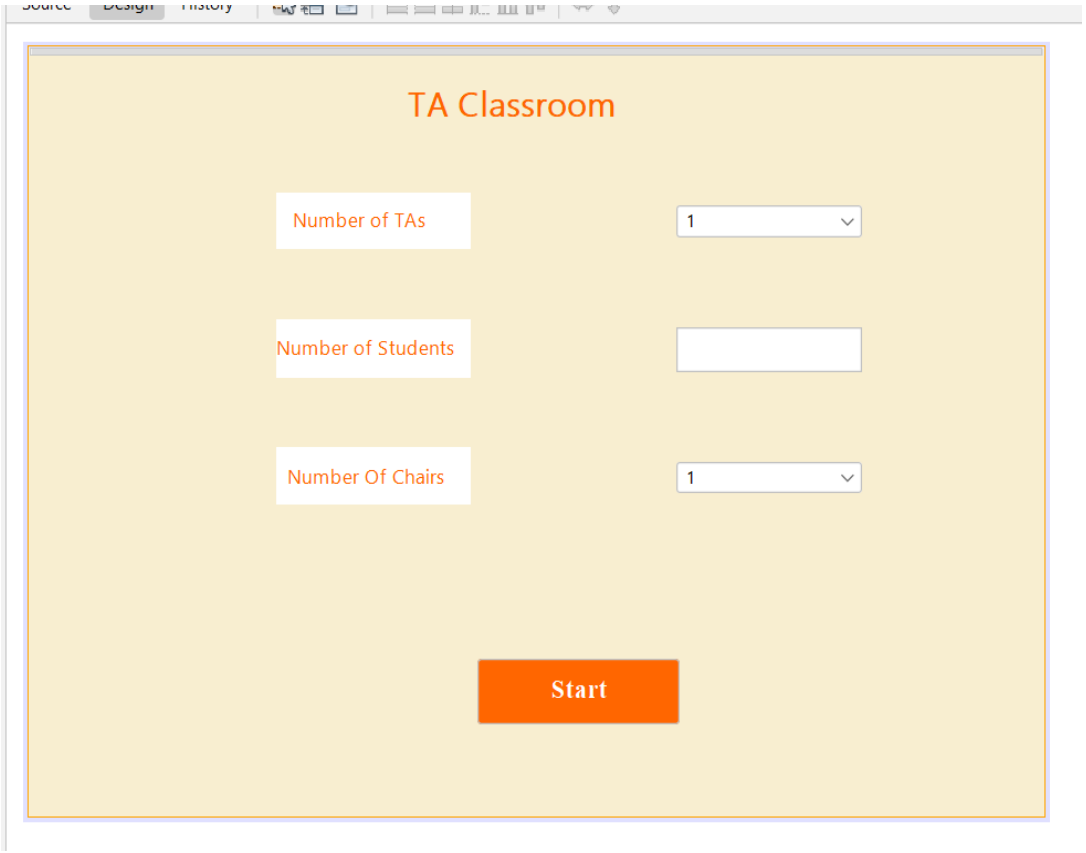
- Handles students who come back later, starting threads for their return.

```java
        ArrayList<Student> backLater = classroom.backLater();
        if (!backLater.isEmpty() ) {
            for (int i = 0; i < backLater.size(); i++) {
            try {
                Student student = backLater.get(i);

                Thread thStudent = new Thread(student);
                exec.execute(thStudent);

                double val = r.nextGaussian() * 2000 + 2000;
                int Delay = Math.abs((int) Math.round(val));
                Thread.sleep(Delay);


            } catch (InterruptedException ex) {

            }
        }
        }

        exec.awaitTermination(20, SECONDS);
        exec.shutdown();

        System.out.println("Classroom is closed");


    }

}
```

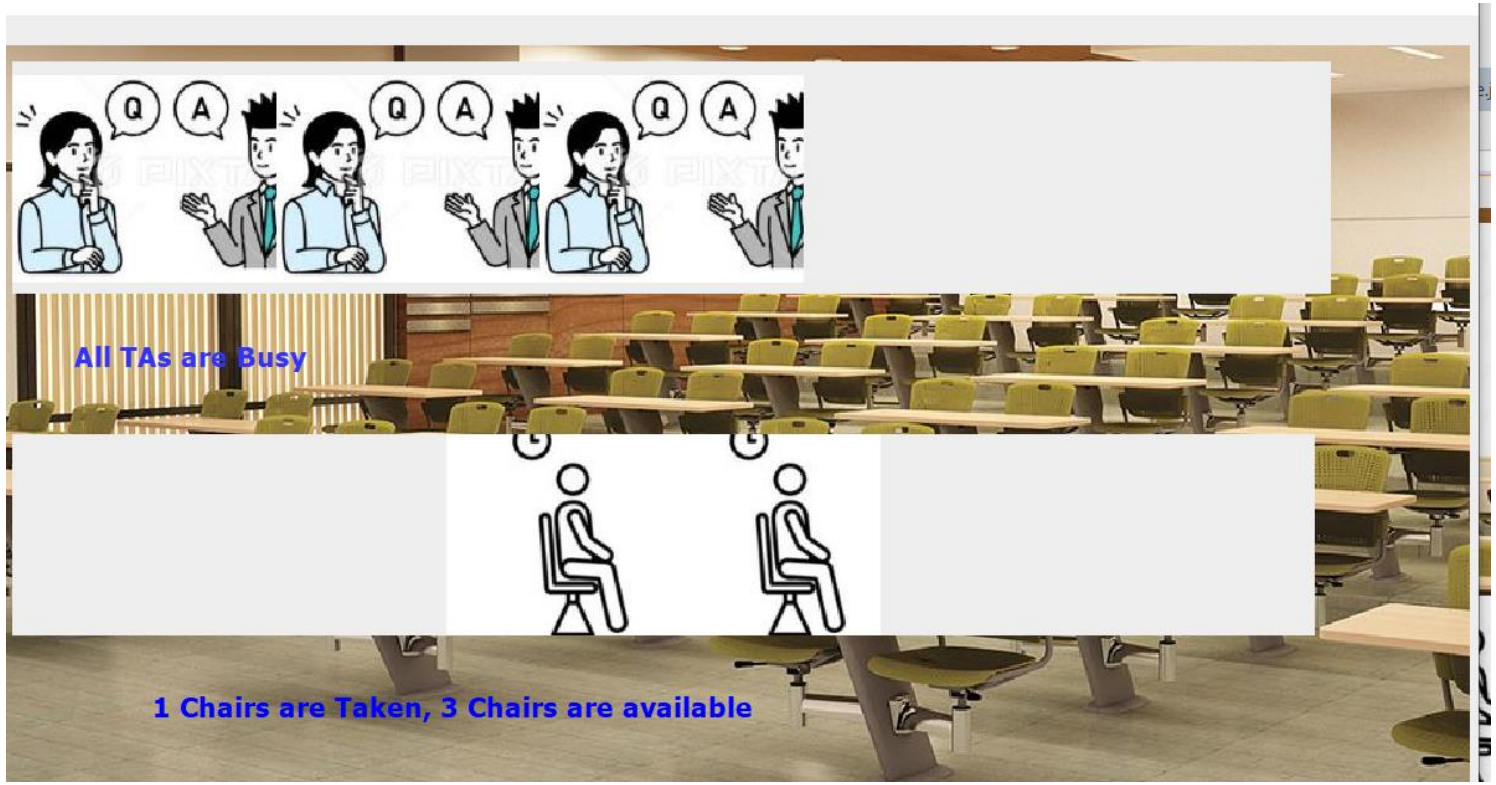
```java
    for (int i = 1; i <= students; i++) {
        try {
            Student student = new Student(classroom);

            student.setStudentId(i);
            Thread thStudent = new Thread(student);
            exec.execute(thStudent);

            double val = r.nextGaussian() * 2000 + 2000;
            int Delay = Math.abs((int) Math.round(val));
            Thread.sleep(Delay);


        } catch (InterruptedException ex) {
            Logger.getLogger(SleepingTAProject.class.getName()).log(Level.SEVERE, null, ex);
        }
    }

    ArrayList<Student> backLater = classroom.backLater();
    if (!backLater.isEmpty() ) {
        for (int i = 0; i < backLater.size(); i++) {
        try {
            Student student = backLater.get(i);

            Thread thStudent = new Thread(student);
            exec.execute(thStudent);

            double val = r.nextGaussian() * 2000 + 2000;
            int Delay = Math.abs((int) Math.round(val));
            Thread.sleep(Delay);


        } catch (InterruptedException ex) {

        }
    }
```

GUI:

- This is an example of components we have used in our project, each component is given to the program by writing number of each them we need in the input form in the GUI.



GUI:

- We insert number of student and Tas and chairs

After this session is started with our components.